Batch-Aware Inventory Engine for Perishable Goods
Python API Reference and Technical Interface Documentation
University of the Cumberlands
MSCS-532-M70: Algorithms and Data Structures
Fall 2026
Hanusha Battula,Shreeya Upendra Gadgil,Kamlesh Kumar,Milan Limbu,
Arpit Sharma

Abstract

This document specifies the callable Python interface of the Batch-Aware Inventory Engine for Perishable Goods. It is organized as an API reference rather than an end-user guide or repository README. The reference identifies modules, classes, public methods, function signatures, parameters, return values, exceptions, side effects, and computational complexity. It also describes the data contracts and consistency rules shared by the interface. The API supports batch lifecycle management, sellable-stock queries, First-Expire, First-Out (FEFO) reservation, order fulfillment and cancellation, and expiration reconciliation. The implementation uses dictionaries and binary min-heaps and is currently an in-memory, single-process Python library.
Keywords: API reference, Python, inventory management, FEFO, min-heap, complexity

API Scope and Conventions

This document describes the Python function and class interface exposed by the project. It is not a REST API: the repository does not implement HTTP routes, network endpoints, authentication, request/response JSON schemas, or a web server. The API is called directly from Python code by importing the relevant modules.
The signatures below reflect the implementation in batch.py, heap_manager.py, inventory.py, and reservations.py. The complexity estimates describe expected behavior for Python dictionaries and binary heaps. Let n be the relevant heap size, d the number of due expiration events processed, k the number of heap entries inspected during FEFO planning, and b the number of batches associated with an order.

Module Overview

Module	Public interface	Responsibility
batch.py	Batch; check_quantity	Batch data model and positive-integer validation.
heap_manager.py	HeapManager	Per-SKU FEFO heaps, global expiration heap, stale-entry validation, and allocation planning.
inventory.py	Inventory; ExpiryReport	Inventory state, batch lifecycle, availability, expiration, and reconciliation.
reservations.py	reserve; fulfill; cancel_reservation	Order reservation, fulfillment, and cancellation workflows.
demo.py	main (demonstration entry point)	Illustrates a typical workflow; not part of the core inventory API.

Shared Types and Data Contracts

Type / field	Contract
date	datetime.date; callers supply calendar dates explicitly.
batch_id	Unique string identifier among currently stored batches.
sku	String product identifier.
quantity	Positive integer at insertion; physical units on hand, including reserved units.
reserved	Integer units held for orders; must not exceed quantity.
available	Sellable units. Zero when expired; otherwise quantity − reserved.
allocation	Dictionary mapping batch_id to units allocated to an order.
ExpiryReport	Dictionary with batch_id, sku, sell_by, and orders fields.

The engine expects the caller to pass today to date-sensitive operations. It does not read the system clock or enforce that successive calls use nondecreasing dates. The default expiry buffer is three days. The sell-by date is calculated as expiry − expiry_buffer; a batch is rejected if sell_by is less than or equal to today.

Core API

Inventory class (inventory.py)

class Inventory(expiry_buffer_days: int = 3) -> None

Creates an empty inventory instance and initializes its indexes, heaps, order ledger, pending expiration reports, and unique version counter.

Parameter	Type	Description
expiry_buffer_days	int	Number of days subtracted from the printed expiry date to calculate sell_by. Defaults to 3; negative values are rejected.

Raises: ValueError if expiry_buffer_days is negative. Complexity: O(1) initialization of empty structures.

Inventory.insert_batch

insert_batch(batch_id: str, sku: str, quantity: int, expiry: date, today: date) -> Batch

Registers a new delivery batch, calculates its sell-by date, adds it to both heaps, and increases the SKU availability counter.

Parameter	Type	Description
batch_id	str	Unique identifier for the delivery batch.
sku	str	Product/SKU identifier.
quantity	int	Positive whole number of physical units.
expiry	date	Printed expiration date.
today	date	Current business date supplied by caller.

Returns: the newly created Batch instance. Raises: ValueError for a duplicate batch ID, invalid quantity (including bool), or a batch whose sell-by date is today or earlier. Side effects: updates batches_by_id, both heap indexes, and available_by_sku. Complexity: O(log n) due to two heap insertions.

Inventory.available

available(sku: str, today: date) -> int

Returns the sellable unit count for a SKU after first processing all expiration events due on or before today. Returns 0 when the SKU has no recorded available stock.

Parameters: sku (str), product identifier; today (date), date used to process expiration. Returns: int. Side effects: may expire batches and queue reconciliation reports. Complexity: O(1) when no expiration is due; O(d log n) when d due events are processed.

Inventory.expire_due

expire_due(today: date) -> list[Batch]

Processes due entries from the global expiration heap. Each live due batch is marked expired, its sellable units are removed from the SKU counter, its version is advanced, and an expiration report is queued when physical quantity is greater than zero.
Returns: list of Batch objects newly expired during this call. Raises: no explicit application exception. Side effects: updates batch expiration state, availability counters, versions, and pending reports. Complexity: O(d log n), excluding report-copy cost.

Inventory.process_expired

process_expired(today: date) -> list[ExpiryReport]

Calls expire_due and returns all pending reconciliation reports, including reports accumulated by earlier operations that triggered expiration. The pending report list is then cleared.
Returns: list of ExpiryReport dictionaries. Each report contains batch_id (str), sku (str), sell_by (date), and orders (dict[str, int]). A report is generated only when the expiring batch has quantity greater than zero. Complexity: O(d log n + r), where r is the number of reports returned.

Inventory.search_batch

search_batch(batch_id: str) -> Batch | None

Looks up a batch by its identifier. Returns the live Batch object when found, otherwise None. Complexity: O(1) average dictionary lookup. Important: the returned object is mutable; callers can change internal state and bypass inventory consistency rules. Treat it as read-only.

Inventory.delete_batch

delete_batch(batch_id: str) -> Batch

Removes a batch from the active batch dictionary and subtracts its currently available units from the SKU counter. Heap entries are not physically removed; they become stale and are skipped when encountered.
Returns: the removed Batch object. Raises: ValueError if the batch does not exist or has reserved units. Side effects: removes the dictionary entry and adjusts available_by_sku. Complexity: O(1) average, not including later stale-entry cleanup.

Batch data model (batch.py)

@dataclass\nclass Batch(batch_id: str, sku: str, expiry: date, sell_by: date, quantity: int, reserved: int = 0, version: int = 0, expired: bool = False, holds: dict[str, int] = ...)

Represents one delivery of one SKU. It is stored in Inventory.batches_by_id and referenced by heap entries and order reservations.

Attribute	Type	Meaning
batch_id	str	Unique identifier.
sku	str	Product identifier.
expiry	date	Printed expiration date.
sell_by	date	Last sellable date boundary used by this implementation; stock becomes unsellable when today >= sell_by.

quantity	int	Physical on-hand units, including reserved units.
reserved	int	Units currently reserved for orders.
version	int	Version token used to invalidate stale heap entries.
expired	bool	True after the batch is marked unsellable.
holds	dict[str, int]	Order ID to number of units held from this batch.

Batch.available property

```
batch.available -> int
```

Returns 0 when expired; otherwise returns quantity − reserved. Complexity: O(1).

check_quantity helper

```
check_quantity(quantity: int) -> None
```

Validates that quantity is a positive Python integer and explicitly rejects bool values. Returns None for valid input. Raises ValueError otherwise. Complexity: O(1). This validation is used by insertion and reservation operations.

Reservation API (reservations.py)

reserve

```
reserve(inv: Inventory, order_id: str, sku: str, qty: int, today: date) -> dict[str, int]
```

Creates an all-or-nothing reservation for qty units of one SKU. Expiration is processed before the availability check. The allocation follows FEFO: batches with the earliest printed expiry date are selected first.

Parameter	Type	Description
inv	Inventory	Inventory instance to modify.
order_id	str	Unique reservation/order identifier.
sku	str	SKU to reserve.
qty	int	Positive whole number of units requested.
today	date	Business date used for expiration processing.

Returns: allocation dictionary {batch_id: units}. Raises: ValueError if the order ID already exists, qty is invalid, available stock is insufficient, or the counter and heap allocation disagree. Side effects: increments each selected batch's reserved value, adds batch.holds entries, decreases available_by_sku, and records inv.orders[order_id]. The allocation is planned before state is committed, so insufficient stock does not produce a partial reservation. Complexity: O(1) for

duplicate/quantity/counter checks plus O(k log n) for heap entries inspected and restored, in addition to any due expiration work.

fulfill

fulfill(inv: Inventory, order_id: str, today: date) -> dict[str, int]

Completes a previously reserved order. Expiration is processed first. If any batch in the order has expired, fulfillment is refused so the caller can cancel and reserve from valid stock. Parameters: inv (Inventory), inventory instance; order_id (str), existing reservation ID; today (date), date for expiration processing. Returns: the fulfilled allocation dictionary. Raises: ValueError if no reservation exists or any allocated batch is expired. Side effects on success: subtracts allocated units from physical quantity, reduces reserved counts, removes reverse holds, invalidates the version of batches that become empty, and removes the order from inv.orders. Availability counter is unchanged because units were removed from availability at reservation time. Complexity: O(d log n + b).

cancel_reservation

cancel_reservation(inv: Inventory, order_id: str) -> dict[str, int]

Cancels an existing reservation and releases its holds. For non-expired batches, released units are added back to the SKU's available counter. For expired batches, units are not returned to sellable inventory.
Parameters: inv (Inventory), inventory instance; order_id (str), reservation ID to cancel. Returns: the cancelled allocation dictionary. Raises: ValueError if the order ID is unknown. Side effects: removes the order from inv.orders, reduces batch.reserved, and removes batch.holds entries. Complexity: O(b), where b is the number of batches in the allocation.

Heap API (heap_manager.py)

HeapManager class

class HeapManager() -> None

Maintains two priority-queue indexes: lots_by_sku, a dictionary of min-heaps ordered by (expiry, batch_id, version); and expiry_events, a store-wide min-heap ordered by (sell_by, batch_id, version).

HeapManager.add_batch

add_batch(batch: Batch) -> None

Pushes one entry into the SKU-specific FEFO heap and one entry into the global expiration heap. Returns None. Side effects: updates both heaps. Complexity: O(log n) for each push; overall O(log n).

HeapManager.live_batch

live_batch(entry: HeapEntry, batches_by_id: dict[str, Batch]) -> Batch | None

Static method. Resolves a heap entry to the current Batch only if its batch ID still exists and its version matches the entry version. Otherwise returns None, identifying a stale entry. Complexity: O(1) average dictionary lookup.

HeapManager.fefo_allocation

fefo_allocation(sku: str, qty: int, batches_by_id: dict[str, Batch]) -> dict[str, int] | None

Plans a FEFO allocation for one SKU by popping the earliest-expiring heap entries, skipping stale, expired, and fully reserved batches, and temporarily retaining valid inspected entries for reinsertion. Returns a batch-to-units mapping when the full quantity can be allocated; returns None when it cannot. For qty <= 0, returns an empty dictionary.

Side effects: heap entries are temporarily popped and then valid inspected entries are restored; it does not change batch reservations or inventory counters. Complexity: O(k log n), where k is the number of entries popped and pushed during planning.

HeapManager.pop_due

pop_due(today: date, batches_by_id: dict[str, Batch]) -> list[Batch]

Pops global expiration events while the smallest sell-by date is <= today. Returns only entries that still resolve to live batches; stale entries are discarded. The caller is responsible for marking returned batches expired and updating counters. Complexity: O(d log n) for d due heap entries popped.

State Consistency Rules

- Reserved units must never exceed physical quantity in a batch.
- Only non-expired, unreserved units may be newly reserved.
- Reservations are all-or-nothing and apply to one SKU per order.
- Expired units must never be returned to sellable availability, including after cancellation.
- Fulfillment must not proceed if any batch allocated to the order has expired.
- Batch deletion is blocked while the batch has active reservations.
- Every heap entry is validated against the current batch ID and version before it is treated as live.

Error and Exception Reference

Condition	Raised by	Exception / behavior
Duplicate batch ID	insert_batch	ValueError
Duplicate order ID	reserve	ValueError
Zero, negative, non-integer, or Boolean quantity	check_quantity via insert_batch/reserve	ValueError
Negative expiry buffer	Inventory constructor	ValueError
Batch sell-by date <= today	insert_batch	ValueError

Unknown batch ID	delete_batch	ValueError
Batch has active reservations	delete_batch	ValueError
Insufficient SKU availability	reserve	ValueError; no partial reservation
Inconsistent counter vs. heap allocation	reserve	ValueError; reservation refused
Unknown order ID	fulfill / cancel_reservation	ValueError
Order contains expired batch	fulfill	ValueError; cancel and re-reserve

Algorithm and Complexity Summary

API operation	**Expected time**	**Reason**
search_batch	O(1) average	Dictionary lookup.
insert_batch	O(log n)	Two heap pushes plus dictionary updates.
delete_batch	O(1) average	Dictionary and counter updates; lazy heap invalidation.
available	O(1) if nothing is due; O(d log n) otherwise	Counter read plus expiration heap processing.
reserve	O(k log n) plus due-expiration processing	FEFO heap inspection and restoration.
fulfill	O(d log n + b)	Due expirations plus allocation traversal.
cancel_reservation	O(b)	Visits each batch in the order allocation.
process_expired	O(d log n + r)	Pops due events and returns reports.

These bounds are expected/structural bounds for the current implementation, not hard real-time guarantees. Heap sizes include stale entries until they are popped. Python dictionary operations are average O(1). The FEFO planner's k can be larger than the number of batches ultimately allocated because stale or unavailable entries may be inspected.

Usage Example

```
from datetime import date\nfrom inventory import Inventory\nfrom reservations import reserve, fulfill, cancel_reservation\n\ntoday = date(2026, 11, 1)\ninv = Inventory(expiry_buffer_days=3)\ninv.insert_batch('B101', 'milk', 20, date(2026, 11, 7), today)\ninv.insert_batch('B102', 'milk', 30, date(2026, 11, 15), today)\n\nprint(inv.available('milk', today))  # 50\nallocation = reserve(inv, 'ORD-1', 'milk', 10, today)\nprint(allocation)                # {'B101': 10}\nprint(fulfill(inv, 'ORD-1', today))
```

The example illustrates the expected call sequence. The returned allocation depends on the batch expiry order and available quantities.

Operational Notes and Limitations

- State is held only in memory and is lost when the Python process exits.
- The implementation is single-threaded and does not provide locks or transaction isolation for concurrent calls.
- Orders support one SKU and whole-order reservation/fulfillment; partial fulfillment and multi-SKU orders are not implemented.
- Expired reservations are reported but are not automatically reallocated.
- Stale heap entries are removed lazily; long-running workloads may benefit from heap compaction.
- Dates are supplied by the caller; chronological progression is not enforced.
- search_batch returns a mutable Batch instance; external mutation can violate invariants.

References

Python Software Foundation. (n.d.). *heapq — Heap queue algorithm*. Python 3 documentation. https://docs.python.org/3/library/heapq.html

Python Software Foundation. (n.d.). *Built-in types*. Python 3 documentation. https://docs.python.org/3/library/stdtypes.html